SF Data Migration Tool — Selectively Agentic Architecture

> **Status.** This describes the system as built. The agent layer, the pre-flight gate and the repair loop are implemented and running; three capabilities were added during implementation that were not in the original plan and are marked as such where they appear — the **deferred-reference and backfill pass** (section 6), **resumable sessions and incremental runs** (section 8), and the requirement that the agent **announce what it wired and propose a next step for what it could not fix** (section 5.5).

1. Division of responsibility

The tool stays a deterministic ETL pipeline: source data is extracted in full to our side, then transformed and loaded into Salesforce. Agents are applied *selectively*, to the two problems that deterministic rules have not solved — getting relationships between objects right, and recovering from load errors. Everything else, including extraction, transformation, loading and ordering, runs exactly as it does today.

Where they are applied, the agents act as a configuration layer: they read system state, decide what the pipeline needs in order to move related data correctly, and then *build that configuration themselves* — which the pipeline executes exactly as it executes hand-written configuration today.

1. **Only the ETL touches data.** Agents read schema, dependency graphs and error summaries. They never read record payloads in bulk, never write to Salesforce, and never modify staged files.
2. **Humans own field-to-field mapping.** Choosing that a source field belongs in a particular Salesforce field remains a human decision, supported by a model-assisted pass that proposes pairings and never writes one.
3. **Agents own relationship correctness and error repair, end to end.** Identifier remapping, identifier maps, load ordering, missing parents, and post-load failures. They do not hand a list of problems back to an engineer to wire up — they wire it up, and report what they did. This is the work that is currently manual, iterative, and the main source of failed runs.
4. **Escalate only what cannot be fixed safely.** Anything inside the bounded remedy set is resolved autonomously. A decision that genuinely needs human judgement — a required parent that is out of scope, an ambiguous default — is raised as a blocker instead of guessed at.
5. **Every change is validated before it is written.** The agent's work is checked against the live Salesforce describe and an allow-list of transform functions. Anything that fails validation is discarded rather than applied, and everything applied is recorded and reversible.
6. **Staged, not streaming.** All selected source data lands on our side before any transform runs, so repair and retry operate on local data and never re-query the source system.
7. **Degrades to today's behaviour.** If Bedrock is unavailable the agent path is skipped and the pipeline runs unchanged. No model call sits on a data-integrity critical path.
8. **A run can be interrupted without being lost.** What has already been written to Salesforce is checkpointed at every phase boundary, so a session that stops part way through continues from there rather than starting again. This was not in the original plan; it became necessary once runs grew long enough that an interruption meant re-loading records that had already landed.

2. Where the agents attach

A migration runs as the numbered sequence below, start to finish, split into the two halves it naturally has: the design-time half on the left, where the migration is configured and nothing has moved yet, and the run itself on the right. Every step but the highlighted ones was already in the pipeline before this work. The highlighted ones are the whole of it — **step 2** where the relationships are resolved and wired, **step 4** where an unresolved blocker stops the launch, **step 11** where a load failure is repaired, and **step 14** where

references that could not be filled during the run are offered back for approval. Following the numbers is the clearest way to see how little of the pipeline changes.



Diagram 1 — The migration procedure end to end. Highlighted steps are the new agent work.

The two halves matter because they carry different risk: everything on the left is reversible at no cost, while everything on the right is touching a live Salesforce org. Note also that step 7 is re-entered from two places: once per object group in the normal loop, and again after a repair — which is why a repair costs one object's transform rather than a restart.

Step 9 does one thing that is not visible in the diagram's arrows. Alongside recording the new Salesforce identifiers it writes a checkpoint of everything that has landed so far, which is what allows a run stopped anywhere on the right-hand side to be picked up rather than repeated. Section 8 covers what that checkpoint contains and, more importantly, what it deliberately leaves out.

Inside step 2 — how the Relationship Agent works

Step 2 is the substance of the design-time half, so it is worth expanding. It proceeds in five passes, each building on the last. The first four work out what is wrong; the fifth is the one that matters operationally, because the agent fixes it rather than reporting it.

1. **Read both sides of the relationship.** On the source side, every field carries a type, and reference fields additionally carry the table they point at — a record's owner field declares that it points at the user table. On the destination side, the Salesforce describe declares which fields are lookups and which objects each lookup accepts. These two facts, put side by side, are what makes dependency discovery possible without reading a single record.
2. **Pair references into dependency edges.** For each source reference field, the agent asks whether the table it points at is also part of this migration, and if so which Salesforce object it becomes. When the answer is yes, that is a dependency edge: the child cannot be loaded correctly until the parent exists. When the answer is no, it is a missing parent — the reference has nothing to resolve against, and it is recorded as a notice whose severity depends on whether the destination field is required.
3. **Derive the load order from the edges.** The set of edges forms a graph, and the graph is sorted so that every parent loads before anything that points at it. This sort is deterministic and stays in code — the agent's job is to make sure the graph is *complete and correct*, particularly that no parent is missing from the selection, because a sort over an incomplete graph produces a confidently wrong order.
4. **Find the edge cases ordering cannot express.** Correct sequencing alone does not make a relationship work. The child mapping must also carry a resolution step that turns the source identifier into the parent's new Salesforce identifier, pointed at the right identifier map — and some shapes cannot be sequenced at all: a record pointing at its own object, two objects pointing at each other, a lookup that accepts several object types, a parent that sits outside the migration's scope. The agent identifies each of these and selects a remedy from the fixed set in section 5.3.
5. **Wire the remedy into the pipeline.** This is the pass that distinguishes the agent from a linter. It does not return a list of things for an engineer to configure — it performs the configuration, then reports what it did. Concretely, it registers the identifier map that the parent mapping will populate during load and attaches the matching lookup transform to the child field, so the child resolves against the parent's new Salesforce identifiers. It pulls a missing parent into the run when the reference cannot work without it. It splits a self-reference or a mutual reference into an insert pass and a follow-up pass that fills the reference once identifiers exist. It writes the resolved parent-first order onto the mappings. It sets a constant or default where a reference points outside scope and a safe default exists. Everything it writes is ordinary pipeline configuration of a kind an engineer would otherwise hand-author, and it is validated against the live Salesforce schema before being saved.

What comes out of step 2 is therefore a **configured migration plus a change log**, not a to-do list. Step 3 is human oversight of work already done: every item can be inspected and reverted individually, and the only things left genuinely outstanding are the blockers the agent deliberately refused to guess at — a

required parent nobody selected, a default with no safe answer. Section 5 covers what it can detect and why that set is deliberately not fixed.

The new steps in detail

Step	When it runs	Where it lives	What it does
2 — Resolve	On request in wizard Step 4 after an initial pass at mapping, and again automatically at launch so anything still unresolved is wired before extraction	A mapping-agent service and endpoint, alongside the existing mappings and compatibility APIs	Registers identifier maps, attaches lookup transforms, pulls in missing parents, sets two-pass handling where needed, and writes the parent-first order — then returns a change log plus any blockers it refused to guess at.
4 — Gate	At launch, before extraction begins; also available on demand from the compatibility step so a blocker is visible before anyone commits to a run	Migration manager, immediately before load-order resolution	Makes no changes and calls no model. Re-validates the configuration as it now stands, refuses to start on unresolved blockers, and reports which references are going to load empty and need a second pass.
11 — Repair	Only when an object reports failed records at step 10	Migration manager, inside the per-group loop after the load call returns	Field-level patches to transform functions and their parameters, followed by a re-transform and re-load of that single object from staged data.
14 — Backfill	After the run, and again whenever a later run supplies the missing side	A deferred-reference service, surfaced as a panel on the run view and a reminder on the dashboard	Lists every reference that loaded empty, with how many rows are now resolvable, and runs the second pass over the ones the operator approves. Nothing is written to Salesforce without that approval.

> **Why these positions and no others.** Step 2 is where a mistake costs nothing to fix, because no data has moved. Step 4 is the last moment at which we can still refuse to start. Step 11 is the only point where Salesforce has told us something we could not have known in advance. Step 14 is the only work that cannot be done during the run at all, because it depends on identifiers that do not exist until the run is over. Anywhere else would place the model between the data and the destination, which the first design rule forbids.

3. How the agent is connected

3.1 Runtime

Each agent is a bounded tool-use loop against the Bedrock Converse API. The model is given a role, a task, and a set of tools; it calls tools to gather facts, then terminates by calling a single submission tool that carries its conclusion. There is no framework in between — the loop is small, and keeping it small is what lets us validate between every turn.

Deployment is a thin service module in the existing FastAPI application. Credentials are resolved server-side and held outside the model's reach: they are captured by the tool executors, never declared in a tool's input schema, so the model can neither see them nor supply them.

3.2 Tools are adapters over logic we already have

This is the reason the agentic layer is a small build rather than a rewrite. The analysis capability already exists in the compatibility engine and is already tested in production runs. The agent does not reimplement any of it — it decides what to inspect and what to conclude.

Tool	Existing capability behind it	What it gives the agent
List mappings	Mapping configuration store	Every mapping in the run with its source table, destination object, operation and identifier map
Describe mapping	Mapping configuration store	The field-by-field detail of one mapping: source field, transform, constant, skip behaviour
Describe source table	Source provider field discovery	Field names, data types, and which table each reference field points at
Describe destination object	Salesforce describe wrapper	Type, length, required, createable, lookup targets, picklist values
Get findings	Deterministic detector	The edge cases the rule set already recognises, with which of them block a launch
Get dependency analysis	Dependency resolver	Topological order, edges, cycles, and missing parents
Get error digest (repair only)	Loader failed-result output	Unique Salesforce error classes with the fields and record counts they affect — never record data
Submit resolution	—	The single, schema-constrained channel through which the agent's decisions leave the loop and enter validation

The detector behind *get findings* is worth separating out from the agent, because it is not a model at all: it is the deterministic rule set, run before any model call, and it is also what the pre-flight gate at step 4 re-runs at launch. The agent receives its output as one input among several rather than as its instructions — the prompt states explicitly that the findings are not exhaustive, which is what section 5.5 depends on.

> **The prerequisite refactor is done.** The dependency resolver and the compatibility report originally loaded mapping configurations from the database only. Both now accept in-memory drafts, which is what lets the agent reason about a change before committing it — and what makes preview mode, where nothing may be written at all, possible.

3.3 Guardrails

Because the agent applies its own conclusions rather than handing them to a person, the guardrails carry more weight than they would in an advisory design. They are what make autonomy safe: the agent can decide freely, but only within a space where every possible decision is already known to be executable.

- **Fixed transform allow-list.** The agent may only configure generic transform functions already registered in the pipeline. Any other function name is rejected — it cannot introduce new executable behaviour, only new configuration of behaviour that already exists and is already tested.
- **Schema verification before write.** Every destination field it touches is re-checked against a fresh Salesforce describe. Unknown or non-createable fields are dropped rather than written.
- **Ordering is recomputed, never trusted.** Whatever sequence the agent arrives at is discarded, and the deterministic resolver is re-run over the configuration it produced. Its real contribution is making the graph complete — ensuring the parent is *in* the selection and wired — while the machine decides the order.
- **Bounded execution.** Turn cap, token cap per call and per session, a capped number of repair attempts per object, and a global feature flag.
- **A concern must come with a next step.** The agent may report something it will not fix, but a report with no suggested action is rejected by the validator before it reaches the interface. It cannot leave a problem on the operator's desk without saying what to do about it.
- **Everything applied is reversible.** Each change is recorded against the session and object that produced it, so a review can revert one item without unpicking the rest.
- **Full audit.** Every decision, validation verdict, and applied change is recorded before it takes effect, with token usage attached.

4. The two flows

Design-time resolution and run-time repair use the same loop, the same validator and the same write path; only the prompt, the tools exposed, and the trigger differ. In both, the agent finishes by writing configuration, not by asking someone to.

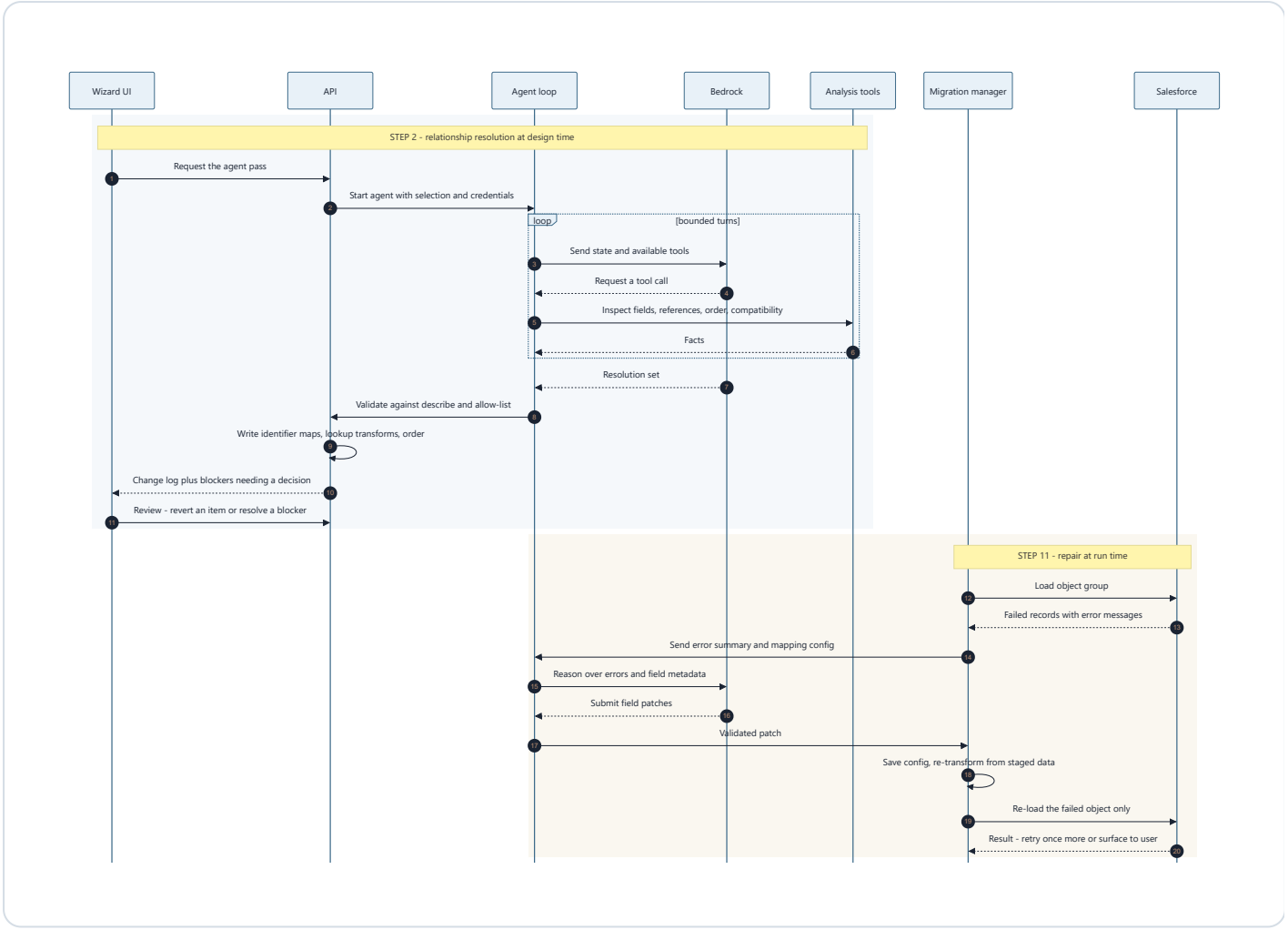


Diagram 2 — Resolution loop at design time; repair loop at run time.

5. Relationship Agent

5.1 The problem it exists to solve

When related records move into Salesforce, Salesforce assigns new identifiers. A child record that pointed at a parent by its source identifier must instead point at the parent's new Salesforce identifier. The pipeline already has the machinery for this — an identifier map populated during load, and a lookup transform that reads it — but somebody has to notice that a given field needs it, register the map under the name the transform will look for, attach the transform to the right field, and ensure the parent loads first.

Today all of that is manual, and it is the single largest cause of failed runs: the symptom appears late — a malformed-identifier error during load, thousands of records in — while the cause is a configuration decision made much earlier. The agent takes over the whole chain, not just the noticing. Detecting the problem and leaving the wiring to a person would keep the expensive half of the work exactly where it is.

5.2 Why the agent generalizes rather than pattern-matching

Identifier remapping is one scenario. It is not the only one, and the important design point is that **we are not building a checklist of known cases**. Deterministic rule checking is exactly where the current implementation already stops: a rule catches the case its author anticipated, and every new client dataset produces relationship shapes nobody anticipated.

The agent is given the whole relational picture rather than a list of tests to run:

- The full source field inventory, including which table every reference field targets.
- The full Salesforce describe for each destination object — lookup targets (including polymorphic targets), required flags, field lengths, and constrained value sets.
- The complete set of objects selected for this migration, and what each one maps to.
- The computed dependency graph, with its edges, cycles, and unresolved parents.
- The naming convention that links an identifier map to the mapping that populates it.

From that surface, the agent reasons about *categories of breakage* — a reference that will not resolve, an ordering that cannot satisfy a dependency, a value that will not survive the destination's constraints — rather than matching a fixed catalogue. When it encounters a relationship shape none of us anticipated, it can still describe the problem in those terms and *fix* it, because the fix is assembled from a small, fixed set of moves that the pipeline already knows how to execute. Novel problem, familiar remedy: that combination is what lets it act on a case nobody wrote a rule for.

5.3 A bounded set of remedies it applies itself

This is what makes autonomous action on open-ended detection safe. Detection is unbounded; action is not. Whatever the agent concludes, the fix it applies must be composed from these primitives, every one of which is something the pipeline already executes today. The agent is choosing and configuring moves, never inventing them.

Remedy	What the agent actually writes
Wire an identifier lookup	Registers the identifier map the parent will populate at load time and attaches the lookup transform, pointed at that map, to the child's reference field
Guard a self-reference	Points the lookup at the object's own identifier map, so a record naming another record of the same type resolves once both have landed
Defer to a second pass	Loads the record without the reference and records which mappings the reference is waiting on, so a backfill pass can fill it later — for self-references and cycles. See section 6
Bring a parent into scope	Adds the parent mapping to the run so its identifiers exist to resolve against
Reorder the load	Writes the parent-first order onto the mappings, as computed by the deterministic resolver
Substitute a constant or default	Sets a fixed target for a required reference whose real target is out of scope
Add a value map	Builds the translation from source vocabulary to the destination's allowed values, read from the describe
Coerce a type	Configures the transform that reformats or bounds the value so the destination accepts it
Make the load re-runnable	Sets the external identifier the destination should match on, so running the same object twice updates rather than duplicates
Raise a concern	Writes nothing, states the problem and what should be done about it, and blocks the launch if the problem is severe enough — the deliberate choice when no safe automatic fix exists

> **The last row is as important as the others.** An agent that always acts is not trustworthy; one that knows when acting would be a guess is. Refusing to fabricate a parent record or invent a default is treated as a first-class outcome, not a failure to resolve — which is precisely why the validator insists such a report names a next step rather than just an objection.

Two of these are less autonomous than they look, deliberately. **Reorder the load** is written by the agent but computed in code: the deterministic resolver produces the sequence and the agent only records it, so the model never chooses an order. And **defer to a second pass** configures the deferral but does not perform the fill — that needs the operator, for the reasons in section 6.

5.4 Illustrative scenarios

The following are examples of what this reasoning surface plus this remedy set covers today — deliberately presented as illustrations, not as the definition of the agent's scope.

Scenario	What the agent recognises	What it does about it
Source identifier copied straight into a lookup field	Destination is a lookup; the mapping copies a raw value with no resolution step	Registers the identifier map and attaches the lookup transform to that field
Parent object not selected for migration	A referenced source table has no mapping in the current selection	Pulls the parent into the run and wires the lookup; falls back to a default record where one is safe, and raises a blocker where neither is
Multi-level chain across three or more objects	Transitive dependency edges in the graph	Wires the lookup at every level and lets the resolver order the whole chain
Self-referencing hierarchy, such as a record naming a parent record of the same type	A dependency edge from an object to itself	Configures a two-pass load: insert first, then fill the self-reference
Circular dependency between two objects	A cycle in the graph, which the resolver cannot order	Defers the least critical edge to a second pass so the rest can be ordered, and flags the choice it made
Circular dependency where only one of the two objects was selected	A reference whose target is absent from this run but does have a mapping available, and whose object points back at the one being migrated	Defers the reference, records which mapping it is waiting on, and tells the operator to add that specific object — rather than reporting an unorderable cycle they cannot act on
A transform named on a mapping that does not exist for that source system	A field configured with a helper belonging to a different source type	Blocks the launch, because the failure would otherwise surface only part way through the transform. The transform picker in the interface is filtered by source type so the choice cannot be made in the first place
Polymorphic reference with several possible targets	A destination lookup that accepts more than one object type	Configures a multi-target lookup with a defined precedence order
Identifier map referenced by the wrong name	A configured map name that does not match the naming convention	Rewrites it to the canonical name derived from the parent mapping
Junction object joining two others	A source table composed almost entirely of reference fields	Wires resolution on both sides of the join and places it last in the order

5.5 Reporting: what it did, and what it could not do

The channel between the agent and the interface is deliberately open. A report carries a severity, a category label, the mapping and field it concerns, a plain-language explanation, and either the change that was made or the decision still required. The category label is not a fixed enumeration, so a relationship problem we have never seen is handled correctly without a deployment: it is resolved and logged if it falls inside the remedy set, or it reaches the person who can decide if it does not.

Three properties of that channel were tightened during implementation, all for the same reason — the operator should never be surprised, either by something the agent did or by something it noticed and left alone.

1. **The agent is asked to review even when the rule set found nothing.** This is the change that matters most. The deterministic checks only cover the cases somebody wrote a rule for, so a configuration with no recognised findings used to skip the model altogether — and an edge case nobody anticipated was, by construction, never reported. The model is now invoked regardless, told explicitly that the findings it has been handed are not exhaustive, and asked to raise anything it recognises as a problem in its own terms. It costs one call per run, and a review call that fails is logged and stepped over rather than stopping the migration.
2. **A concern must name a next step.** A report with no suggested action is rejected before it reaches the interface. The point of surfacing a problem the agent will not fix is to hand over something actionable; an observation the operator cannot act on is worse than silence, because it invites the run to be launched anyway.
3. **Applied changes are announced as they happen, not discovered afterwards.** When the agent wires something at launch it emits a plain sentence into the live run view before extraction starts — "*Agent wired 3 lookup fields on Account before extraction started*" — and the equivalent for anything it flagged. Previously this was only visible by opening the change log after the fact, which meant the first time an operator learned the agent had touched their configuration was usually when something else went wrong.

In the interface these land in one panel per session: what was wired, prominently and with a revert control on each item, and separately the concerns with their suggested actions. Both are stored against the session rather than living only in the response that produced them, so they survive a page reload, a restart, and a resume on another machine.

> **Compounding benefit.** Because every report is audited with its category, the categories that recur across clients become visible. Those graduate into deterministic checks in the rule set — cheaper, instant, and no model call required. The agent effectively acts as a discovery mechanism that keeps

> hardening the deterministic layer beneath it; two of the checks listed in section 5.4 arrived exactly that way.

6. Deferred references and the backfill pass

Not in the original plan. "Split into two passes" appeared there as a single line in the remedy table; it turned out to need a mechanism of its own, an approval step, and a place in the interface.

6.1 The references that ordering cannot save

Most relationships are solved by loading the parent first. Some cannot be. A record pointing at another record of its own type, or two objects that point at each other, has no parent-first sequence — whichever one loads first, its reference has nothing to resolve against. The only correct answer is to insert the record without that reference and fill it once both sides have Salesforce identifiers.

So during the run, a deferred reference simply loads empty and the row is set aside. The record itself lands normally, with everything else about it intact. What is set aside is the identifier the reference was supposed to carry, along with which mapping it was waiting on.

6.2 Why it needs approval rather than happening automatically

Filling those references means a second write to Salesforce, updating records that have already landed. That is a different risk category from the run the operator authorised, and the run's outcome does not depend on it: the records are there and correct apart from one field. So the pass is offered rather than performed. Each deferred reference becomes an item showing which object and field it concerns, how many rows are waiting, and — recomputed each time it is viewed — how many of those could actually be filled right now. Approving an item and running it are separate actions, so nothing is written because a button was clicked in passing.

6.3 The case the original design missed

> The mechanism above only engaged when *both* sides of a mutual reference were part of the same run. In practice an operator selects one object — `Account`, whose references and `User`'s point at each other — and nothing happened at all: no deferral, no item to approve, no explanation. The cycle was invisible because half of it was out of scope.
>
> The rule set now recognises this shape specifically: a reference whose target is absent from the run, but which *does* have a mapping available and whose object references back at the one being migrated. The reference is deferred as before, and it records the name of the mapping it is waiting for — so the item reads "*add* `User` *to this migration and run it*" rather than the unhelpful "*migrate*

> *the object it points at first*", which is advice the operator has no way to follow when they do not know which object that is.

6.4 Partial fills

A backfill pass often cannot fill everything, because some of the referenced parents still have not been migrated. This case is handled explicitly rather than optimistically: the rows that were filled are recorded, the item returns to the waiting state carrying only the rows that are still outstanding, and its running total accumulates across passes. An earlier implementation marked a partial fill as complete and silently discarded the remainder, which is the kind of defect that surfaces months later as a set of records nobody can explain.

Because an item can outlive the run that created it, the dashboard carries a standing reminder for deferred references that have *become* fillable — the counterpart migrated in some later session, and the item that was stuck is now ready. Without that, a deferral raised in January is only found by someone who thinks to go looking for it.

7. Repair Agent

7.1 What it works from

The loader already extracts unique error messages from Salesforce's failed-record output and returns them per object. The repair agent receives that error summary, the object's mapping configuration, and the destination field metadata. It never sees record data — only error text and schema — which keeps the same data-handling posture as the Relationship Agent.

Salesforce error messages are self-describing enough to reason from, and the same bounded remedy set from the previous section applies. Common classes and their typical resolutions:

Error class	Typical resolution
Value in the wrong format for its type, commonly dates	Reformat with the correct input pattern
Value longer than the destination allows	Bound it to the described length
Value outside a restricted value set	Add a value map built from the destination's own allowed values
Malformed identifier	A missing resolution step — handed back to relationship logic rather than patched locally
Required field empty on some records	Supply a default, or escalate as a blocker if no safe default exists
Duplicate detection, permission and access failures	Not repairable by configuration; surfaced to the user without retry

7.2 Why staging makes repair practical

Because every selected record already sits on our side before any transform runs, a repair is cheap: the failed object is re-transformed from local data and re-loaded on its own. The source system is not re-queried, successfully loaded objects are untouched, and the identifier map entries already written remain valid. Repair on a streaming pipeline would mean re-extracting and reasoning about partial state; here it is a local, scoped, repeatable operation.

> **Stopping conditions.** A capped number of repair attempts per object per run. Abort immediately if an attempt does not reduce the failure count, if the error class is on the non-repairable list, or if the same patch has already been tried. A run that exhausts repair finishes in the existing completed-with-errors state with the unresolved errors attached — the agent can improve an outcome but never worsen one.

8. Resumable sessions and incremental runs

Not in the original plan. It was added once runs grew long enough that losing one to an interruption meant re-loading records that had already landed in the customer's org.

8.1 What a checkpoint is, and what it is not

A checkpoint is the record of **what has already been written to the destination**. That is the whole definition, and it is what makes the contents easy to reason about: the identifier map, the mapping configurations exactly as they were run including anything the agent rewired, the change log, the per-

object progress counters, and the list of objects that finished. It is written after extraction, after each object group that loads cleanly, on pause, on stop, on failure and at completion.

Two things are deliberately excluded. **Credentials** are stripped before the checkpoint leaves the machine, which means restoring a session onto a host that has never held them reports that they are required rather than pretending it can proceed. And **the staged source extract** is not uploaded — it is large, and it is reproducible from the source system at any time. What is stored instead is a fingerprint of it, which is enough for a resume to tell whether the local copy still matches the one the checkpoint was taken against and to re-extract only when it does not.

> **A checkpoint failure never fails a migration.** Saving is best-effort and logged. The worst outcome of an unreachable bucket is a run that cannot be resumed — never a run that stops because it could not record that it was going fine.

8.2 What resuming actually skips

Resuming restores the identifier map and the progress records, then makes two decisions. Objects already recorded as finished are dropped from the plan entirely. Extraction is skipped if the stage fingerprint still matches, and re-run if it does not. Within an object that was only part way through, the transform skips rows whose identifiers are already in the map — so the granularity of a resume is a row, not an object, and an interruption two thirds of the way through a large table costs the remaining third.

8.3 Adding objects to a session that has already run

A session's selection is append-only: objects can be added to it, a plan can be previewed showing what a run would execute against what it would skip, and only the outstanding work can be run. Removal is not offered, because the records are already in Salesforce and narrowing the selection would narrow the identifier map that later objects resolve against — quietly breaking references that currently work.

> **This is what closes the loop with section 6.** Migrate `Account` today and its `User` reference is deferred. Add `User` to the same session next week and run the remaining work; the deferred item recomputes, reports itself as fillable, and the backfill pass completes the reference. The two features were built separately and this is the case that needs both.

9. State and interfaces

9.1 New state

Four new tables. The first records **agent activity**: every row captures the session and object it concerns, the agent and phase, the change that was made, whether it was applied, the prior value it replaced, the rationale and the severity. Because the agent writes rather than suggests, this table does three jobs at once — it is the audit trail, the mechanism that makes any individual change reversible, and the cost ledger. It also holds the concerns the agent raised but did not act on, stored unapplied, which is what lets them survive a restart and feeds the scenario-frequency analysis in section 5.5.

The others follow from the two capabilities added during implementation: one tracks each **deferred reference** and its progress through waiting, approved and filled; one indexes the **session checkpoints** held in object storage so they can be listed without a network round trip; and one caches **schema describes** per connection, so a wizard session and an agent run inspecting the same object do not each pay for it.

9.2 Existing state the agents depend on

- **Mapping configurations** — the only design-time write target. The agent adds transform functions, their parameters, and load priorities to the same structure the wizard already saves; nothing structurally new is introduced, which is why its output needs no special handling downstream.
- **Identifier maps** — populated during load, read by lookup transforms. The agent registers them as part of wiring a relationship, always deriving the name from the shared naming helper rather than composing one. Hand-built names are the most common relationship defect in the current process, and the validator checks for exactly this before anything is written.
- **Job and batch records** — repair retries reuse the same job identifiers so run history stays coherent and a repaired run reads as one run.

9.3 Interfaces

Interface	Purpose
Resolve endpoint	Takes the current selection and credential references. Runs the agent, validates its conclusions, writes the resulting configuration, and returns a change log, the outstanding blockers, and token usage.
Preview mode on the same endpoint	Identical analysis with the write suppressed. Useful for a first run on an unfamiliar client dataset, and for testing.
Pre-flight endpoint	Runs the launch gate without starting anything, so blockers and deferred references are visible from the wizard before a run is committed to.
Change log endpoint	Serves what the agent applied and what it flagged, per session — the source for the panel on the run view.
Revert endpoints	Restores the prior value from the audit record, for one change or for a whole agent run. Reversal never needs the model to be reachable.
Deferred reference endpoints	List the outstanding references, record an approval or a decline, and run an approved pass. Approving and running are separate calls.
Checkpoint endpoints	Report whether checkpointing is configured and the bucket reachable, list saved sessions, and restore or resume one.
Session plan endpoints	Add objects to a session, preview what a run would do, and run only the outstanding work.
Order advice endpoint	Narrates a migration order over the dependency graph for the hierarchy view. Advisory: the loader continues to use the computed order, and the response says whether the two agree.

Interface changes in the web application:

- An agent action in the mapping step, plus a model-assisted pass that drafts the field pairings — both the ones whose names already agree and the ones only a reading of the labels would find. Both propose; neither writes.
- Selection on both sides — source tables and destination objects — with a tree of everything the chosen Salesforce objects depend on above them, tiered by whether a dependency is required, merely useful, or part of a cycle. Any node opens a panel describing that object's fields and types, so the operator does not have to leave for Salesforce setup to answer "what is actually on this object".
- A change-log panel rather than an approval queue: each item states what was wired, on which field, and why, with a revert control. The default is that the work is done; the user is auditing it, not

completing it.

- A separate, prominent list of the concerns the agent declined to guess at, each with the next step it suggested. These are the only items that actually need a human decision, and they should not be buried among the changes it already handled.
- A live line in the run view at the moment the agent wires something, so the change is announced before extraction rather than discovered afterwards.
- A deferred-reference panel on the run view with per-item approval, and a standing reminder on the dashboard for items that have become fillable since.
- A per-object status report that works for a finished or interrupted session as well as a running one, since a resumed session needs to show what landed before the interruption.
- A repair state on the progress view, so an in-flight repair and its rationale are visible while it happens rather than only in the final error list.

10. Delivery status

The sequence below is the one the work was planned against, in order, with what it turned into. Stages 1 to 9 are the original plan; stages 10 to 13 are the work that was added during implementation and is described in sections 5.5, 6 and 8.

#	Stage	Complete when	Status
1	Bedrock client and configuration	A round trip to the target model succeeds from the application environment	Delivered
2	In-memory configuration refactor	Dependency resolution and compatibility checks run on draft configurations without database writes	Delivered
3	Tool adapters	Each tool is independently exercisable against real source and destination credentials	Delivered
4	Agent loop and validator	A deliberately malformed conclusion is rejected rather than written	Delivered
5	Relationship agent in preview mode	The change log for a known multi-object set matches what an experienced engineer would wire by hand	Delivered
6	Audit trail and revert	Every change is recorded with its prior value and can be reversed individually	Delivered
7	Enable writes, plus the wizard change-log view	A run wired entirely by the agent completes without hand-editing a mapping	Delivered
8	Repair agent	A seeded format failure self-heals on the first retry	Delivered
9	Repair visibility in the run view	Repair state and rationale are visible live	Delivered
10	**Added:** deferred references and the backfill pass	A cycle loads without error, and the reference is filled from an approved second pass	Delivered
11	**Added:** cycles where only one side was selected	Migrating one object of a mutually referencing pair defers correctly and names the object to add	Delivered
12	**Added:** resumable sessions and incremental runs	An interrupted run continues without re-loading what already landed; objects can be added to a finished session	Delivered
13	**Added:** review-always, suggested actions, live wiring summaries	A configuration with no recognised findings still gets a model review, and anything raised carries a next step	Delivered

What the sequence was worth

Two ordering decisions did most of the work and are worth keeping for similar projects.

- **Autonomy was switched on at stage 7, not stage 5.** The agent ran in preview for two stages first, so its change log could be compared against hand-wiring on real client data before it was permitted to write anything — and the revert path existed before the first write did.
- **Stages 1 to 7 carried no runtime risk.** None of them execute during a migration, so they could be built around live client work rather than blocking it. Repair was the first component to run inside a live migration, which is why it landed last and sits behind both a feature flag and its own attempt cap.

Stage 5 was the only empirical stage and, as expected, the one whose duration could not be estimated in advance — prompt and tool-surface iteration converges when it converges. The four added stages were not foreseeable from the plan: each one came from running the system against a real dataset and finding a case the design had described in a single line without noticing it needed a mechanism.

11. Operational posture

Concern	Handling
Bedrock unavailable or throttled	Adaptive retry, then fall through to the deterministic path; the run proceeds as it does today
What the model receives	Schema and error text only — field names, types, allowed values, and Salesforce error messages. No record payloads, no credentials, no attachments
Credential handling	Resolved server-side by reference; never declared in a tool schema, never placed in a prompt, and stripped from a checkpoint before it leaves the machine
What leaves the machine at all	Model calls carry metadata. Checkpoints carry the record of what was written to Salesforce, encrypted at rest. The staged source extract stays on the host and is never uploaded anywhere
Checkpoint storage unavailable	Saving is best-effort and never aborts a run. The consequence of an unreachable bucket is a session that cannot be resumed, not one that fails
Cost	A resolution pass is a modest number of tokens against a schema, not against data; a repair is smaller still. Both are capped and logged per call, so spend is measurable per migration from day one. Reviewing when nothing was found adds one call per run
Latency	Resolution is user-initiated at design time and takes tens of seconds. Repair adds one model call and a scoped re-transform to a run that had already failed
Non-determinism	Deterministic decoding settings, and every conclusion validated against live schema before it is written. Two runs may reach the same outcome by different routes; neither can write an invalid configuration
Wrong-but-valid changes	The residual risk of autonomy: a change that passes validation but is not what the client wanted. Contained by announcing each change as it is applied, the change log, per-item revert, and the pre-flight gate at step 4 — not by asking a human to approve every item, which is the review nobody performs carefully
Second writes to records already loaded	The one place where explicit approval *is* required, because filling a deferred reference updates records the operator has already accepted. Approving and running are separate actions

Concern	Handling
Auditability	Every decision, verdict and applied change is attributable to a session, an object, and a model version, with the value it replaced. Concerns the agent raised without acting on are stored the same way

---

Staged pipeline: extract all selected data to our side, then transform, load, and record identifier mappings. Agents resolve relationships and write the configuration themselves; the deterministic pipeline executes it. What cannot be resolved during the run — a reference waiting on identifiers that do not exist yet — is offered back for approval rather than guessed at, and what has already landed is checkpointed so an interrupted run continues instead of repeating.